数据结构研讨课报告 1

学号:2453359 姓名:李欣洋

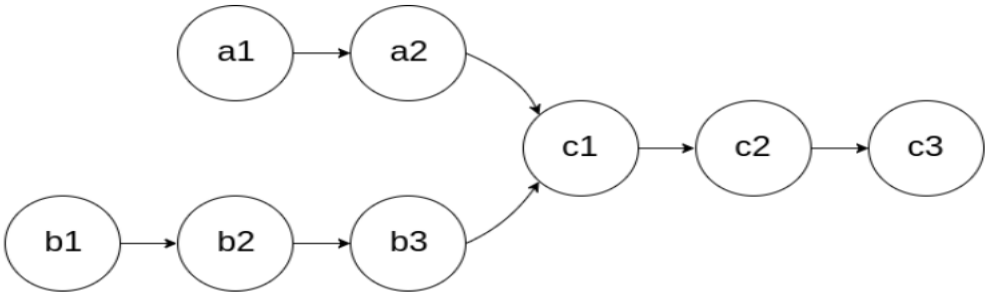
一.QS:给定两个单向链表的头指针,判定它们是否相交(假设两个链表均不带环)

(1) 请画出链表相交示意图

(2) 给出算法思想,并分析时间复杂度

(3) 如何求出两个单链表的第一个交点

1.链表相交示意图



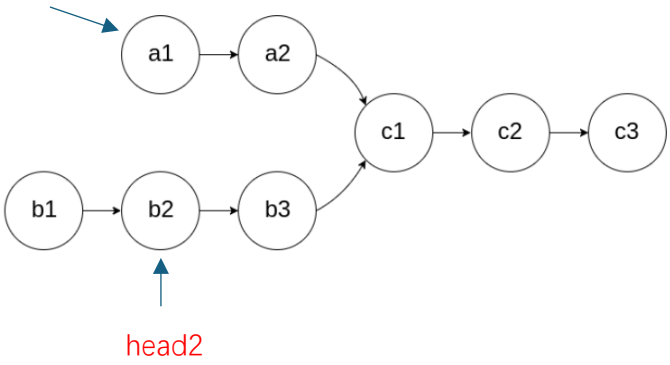
实现链表的数据结构:

```
struct ListNode
{
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(NULL) {}
};
```

2.1 算法 1:双指针+求解两个链表的长度差

思路简述:首先设置两个指针,在此取名为 head1 和 head2,分别指向 a1 和 b1,然后采用两个循环,遍历两个链表,使用变量记录遍历次数来反映链表的长度,然后找到较长者,让其头指针,先移动(较长链表的长度-较短链表的长度)步

head1



此时两个指针的位置是对称的,这时再去同时遍历这两个链表,都从 head1 和 head2 开始,如果他们有相交的话,他们走到交点的距离是相等的,当两个指针相等时,返回 head1 或者 head2,就找到了两个链表的交点.

2.1_1 算法的代码实现:

```
// 步骤1: 计算链表A 的长度
       int lengthA = 0;
       ListNode* head1 = headA;
       while(head1 != nullptr)
       {
           lengthA++;             // 记录链表A 的节点个数
           head1 = head1->next;    // 移动到下一个节点
       }
//步骤 2:计算链表 B 的长度,省略
//步骤 3:将两个指针复原(恢复至初始情况),省略
//步骤 4:让较长者先进行移动 fbs(lengthA-lengthB)步
// if(lengthA > lengthB)  // 如果链表A 更长

       {
           int diff = lengthA - lengthB;  // 计算长度差
           for(int i = 0; i < diff; i++)
           {
               head1 = head1->next;  // 链表A 的指针先走diff 步
           }
       }
//若 B 链表更长,同理,省略
//步骤 5:两个指针同时移动，直到相遇或都到达末尾

       while(head1 != nullptr && head2 != nullptr)
       {
           if(head1 == head2)  // 如果两个指针指向同一个节点，说明找到交点
           {
               return head1;//找到交点,返回交点
           }
```

```
            head1 = head1->next;  // 两个指针同步向后移动
            head2 = head2->next;
        }
// 步骤6: 如果没有找到交点，返回nullptr
            return nullptr;
```

2.1_2 逻辑结构: 单链表，属于线性表的链式存储。每个节点逻辑上只有一个直接后继（除了最后一个节点）。

2.1_3 存储结构: 链式存储结构。通过结构体 ListNode 实现，每个节点包含数据域（val）和指针域（next）。节点在内存中的物理位置可以是不连续的，通过指针 next 将各个节点串联起来形成链表。这种存储方式的优点是插入和删除操作灵活，不需要移动大量元素；缺点是不支持随机访问，查找某个元素必须从头节点开始顺序遍历。

2.1_4 时间复杂度分析

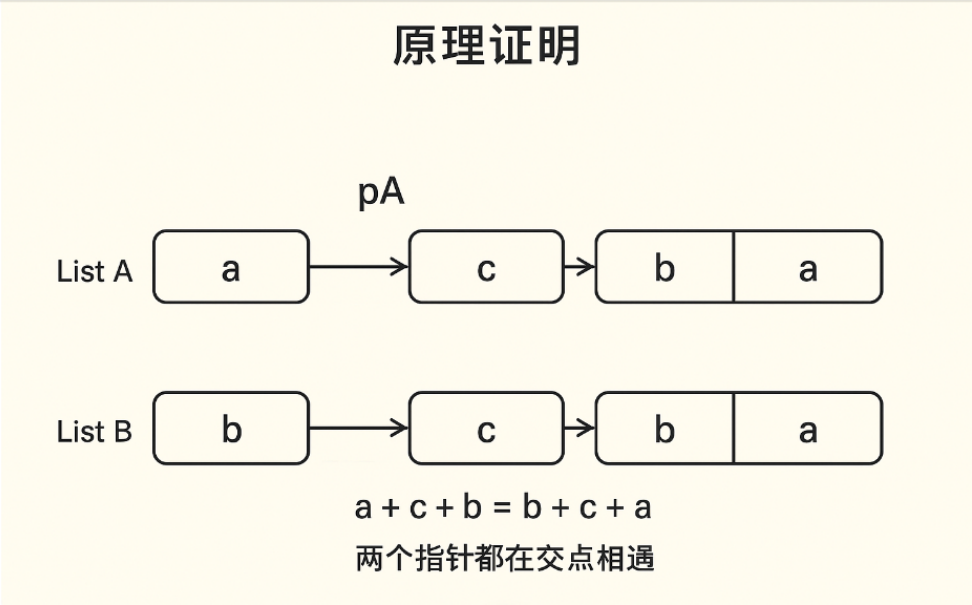
该算法的时间复杂度为 O(m + n)，其中 m 和 n 分别是两个链表的长度。具体分析：首先需要遍历链表 A 一次计算长度（O(m)），然后遍历链表 B 一次计算长度（O(n)），接着较长的链表需要先移动若干步（最多 O(max(m,n))），最后两个指针同步移动直到相遇或到达末尾（最多 O(min(m,n))）。因此总的时间复杂度为 O(m) + O(n) + O(max(m,n)) = O(m + n)。空间复杂度为 O(1)，只使用了常数个额外指针变量。

2.2 算法 2:双指针法

思路简述:创建两个指针 pA 和 pB 分别从 headA 和 headB 开始遍历。当指针不为空时正常向后移动；当指针遇到空节点（到达链表末尾）时，就跳转到另一个链表的头节点继续遍历。两个指针同步移动，最终要么在相交节点相遇，要么都变为 nullptr（无交点情况）。

原理证明:

- 假设链表 A 的非相交部分长度为 a，链表 B 的非相交部分长度为 b，相交部分长度为 c
- 指针 pA 的行走路径：a + c + b（走完链表 A 后，从链表 B 头开始走）
- 指针 pB 的行走路径：b + c + a（走完链表 B 后，从链表 A 头开始走）
- 因为 a + c + b = b + c + a，所以两个指针走过相同的距离后必然在相交节点相遇
- 如果没有交点（c=0），则两指针都会在走完 a + b 的距离后同时变为 nullptr，循环结束



2.2_1 算法的代码实现:

```
//关键部分的代码,先使用两个指针指向两个链表的头节点，然后同时遍历两个链表
        ListNode* tempa = headA;
        ListNode* tempb = headB;
        while(tempa != tempb)
        {
            //当tempa 到达链表末尾时，将其重定向到另一个链表的头节点
            if(tempa == nullptr)
            {
                tempa = headB;
            }
            else
            {
                tempa = tempa->next;
            }
            //当tempb 到达链表末尾时，将其重定向到另一个链表的头节点
            if(tempb == nullptr)
            {
                tempb = headA;
            }
            else
            {
                tempb = tempb->next;
            }
        }
        return tempa;//两个指针相等了,返回其一即可
```

2.2_2 逻辑结构: 特殊的 "Y 型" 结构,两个链表在某个节点处合并，形成一个分叉的线

性结构。相交后的部分由两个链表共享

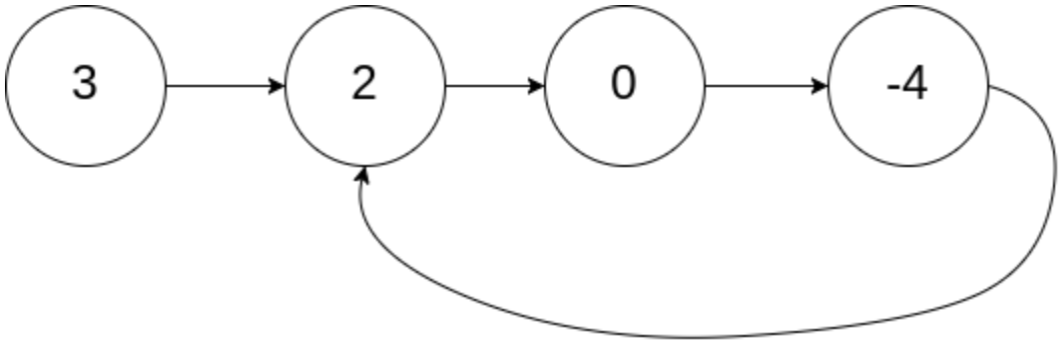
2.2_3 存储结构：链式存储结构。通过结构体 ListNode 实现，每个节点包含数据域（val）和指针域（next）。节点在内存中的物理位置可以是不连续的，通过指针 next 将各个节点串联起来形成链表。这种存储方式的优点是插入和删除操作灵活，不需要移动大量元素；缺点是不支持随机访问，查找某个元素必须从头节点开始顺序遍历。

2.2_4 时间复杂度分析：时间复杂度：O(m + n)，其中 m 和 n 分别是两个链表的长度。

- 在最坏情况下，每个指针最多遍历两个链表的总长度。具体来说：
- 如果有交点：指针 pA 走 a + c 步后到达链表 A 末尾，然后从链表 B 头开始走 b 步到达交点，总共 a + c + b 步；指针 pB 同理走 b + c + a 步
- 如果无交点：每个指针都会遍历完两个链表，即 m + n 步后同时为 nullptr
- **空间复杂度**：O(1)，只使用了两个指针变量，没有使用额外的存储空间。
- 相比于 2.1 的"先计算长度再对齐"的方法，这个算法虽然时间复杂度相同，但代码更简洁优雅，只需要一次遍历即可完成。

二.如何判断一个单链表内是否有环?若有环,找到入环的第一个节点

1.有环单链表示意图:



2.实现链表的数据结构:

```
struct ListNode
{
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(NULL) {}
};
```

3.算法思路简述：采取算法——快慢指针法

原理：类比操场跑步：两个人在环形跑道上，跑得快的人最终会追上跑得慢的人,如果

是直线跑道（无环），快的人会先到达终点

算法思路 1. 创建两个指针：慢指针(slow)每次移动一步，快指针(fast)每次移动两步 2. 如果链表有环，快指针最终会追上慢指针（相遇）3. 如果链表无环，快指针会先到达链表末尾（nullptr）

4.算法的具体代码实现

```
/ // 初始化快慢指针
    ListNode * slow = head;        // 慢指针，初始指向头节点
    ListNode* fast = head->next;  // 快指针，初始指向头节点的下一个节点

    // 当快慢指针不相等时，继续循环
    while(slow != fast)
    {
        // 如果快指针或快指针的下一个节点为空，说明到达链表末尾，无环
        if(fast == nullptr || fast->next == nullptr)
        {
            return false;
        }

        // 慢指针每次移动一步
        slow = slow->next;

        // 快指针每次移动两步
        fast = fast->next->next;
    }

    // 跳出循环说明 slow == fast，快慢指针相遇，链表有环
    return true;
```

5.逻辑结构：带环的单链表，属于非线性结构（因为有环）

6.存储结构：链式存储，通过指针连接，环的形成是因为某个节点的 next 指针指向前面的某个节点

7.时间复杂度分析：

- **O(n)**，其中 n 是链表的节点数。
- 如果无环：快指针会在 n/2 步内到达末尾
- 如果有环：设环外部分长度为 a，环的长度为 b
- 慢指针进入环时，快指针已经在环内
- 在环内，快指针每次比慢指针多走一步，相当于快指针在追赶慢指针

- 最多经过 b 次循环，快指针就能追上慢指针

空间复杂度:O(1)，只使用了两个指针变量，没有使用额外的存储空间。

三.医院看病排队管理

1.情景考虑

(1)医院的普通门诊,按照先来后到的顺序进行挂号

(2)医院的急诊,这时除了挂号之外,还需要考虑的每个急诊病人的病情严重程度,我们应该把病情严重程度最重的病人的治疗优先级提到最高

2.数据结构考虑:

(1) **普通门诊 - 使用 queue（队列）**

- **原因**：普通门诊遵循"先来先服务"原则（FIFO - First In First Out）
- **特点**：按挂号顺序依次就诊，公平合理
- **操作**：push() 挂号入队，front() 查看队首，pop() 叫号出队

(2) **急诊 - 使用 priority_queue（优先队列）**

- **原因**：急诊需要根据病情严重程度优先处理
- **特点**：紧急程度高的患者优先就诊，相同紧急程度则先来的优先
- **操作**：push() 挂号入队（自动排序），top() 查看优先级最高的，pop() 叫号出队
-

3.具体数据结构的代码实现:

(1)患者结构体:

```
struct Patient
{
    string name;          // 患者姓名
    int id;               // 患者编号（挂号顺序）
    string symptom;       // 症状描述
    int urgencyLevel;     // 紧急程度（1-5，5 最紧急，仅急诊使用）
    string registTime;    // 挂号时间
    }
```

(2)医院排队系统类:

```
class HospitalQueueSystem
{
private:
    queue<Patient> normalQueue;          // 普通门诊队列（FIFO）
    priority_queue<Patient, vector<Patient>, EmergencyComparator>
emergencyQueue;  // 急诊优先队列
    int patientCount;                   // 患者总数计数器
    int normalCallCount;                // 普通门诊叫号计数
    int emergencyCallCount;             // 急诊叫号计数
}
```

4.一些重要的代码块:

因为编译器无法对自定义数据类型进行直接的比较,所以需要重载运算符,来为优先级队列实现一个比较器

```
// 急诊患者的比较器（用于优先队列）
// 紧急程度高的优先，紧急程度相同则先来的优先
struct EmergencyComparator
{
    bool operator()(const Patient& p1, const Patient& p2)
    {
        // 如果紧急程度不同，紧急程度高的优先（返回 false 表示 p1 优先级高）
        if(p1.urgencyLevel != p2.urgencyLevel)
        {
            return p1.urgencyLevel < p2.urgencyLevel;  // 数字大的优先
        }
        // 紧急程度相同，编号小的（先来的）优先
        return p1.id > p2.id;
    }
};
// 普通门诊挂号
    void registerNormal(string name, string symptom)
    {
        patientCount++;
        Patient p(name, patientCount, symptom);
        normalQueue.push(p);

    }
```

5.逻辑结构和存储结构:

普通门诊队列:

(1) 逻辑结构: 线性表（队列）

(2) 存储结构: 使用数组存储队列元素，用两个指针 front 和 rear 标记队首和队尾。

急诊优先级队列:

(1) 逻辑结构:一个支持自动排序存储元素的队列,且与普通队列不同,支持在双端进行操作

(2) 存储结构:数据应该是被存储在堆区,因为每一次的排序都要从中间拿取元素,不会是一个连续的内存来进行存储,因此数据是存储在堆区,内部由指针进行管理